Experiment-9

Aim: Implement Hashing.

1. Hashing

Definition:

Hashing is a technique to store and retrieve data efficiently using a **hash function** that maps a **key** to a **hash value (index)** in a **hash table**.

Advantages:

- Fast search, insertion, and deletion (average O(1))
- Efficient for large datasets

Key Terms:

- **Hash Function:** Computes index from a key
- **Hash Table:** Array where data is stored based on hash value
- **Collision:** When two keys produce the same hash index

2. Types of Hashing

1. **Direct Hashing:** Key directly used as index
2. **Modulo Hashing:** Index = key % table_size
3. **Collision Handling:**
    - **Chaining:** Store multiple elements at same index using a linked list
    - **Open Addressing:** Find next available slot (linear probing, quadratic probing, double hashing)

3. Algorithm for Hash Table (Chaining)

Insertion:

1. Compute hash index: index = key % table_size
2. If index is empty → insert element
3. If index is not empty → append element to linked list at that index

Search:

1. Compute hash index
2. Traverse the linked list at that index to find the key

Deletion:

1. Compute hash index
2. Traverse the linked list at that index

Dept of AIML

3. Remove the key if found

Program:

```
# Node for chaining
class Node:
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.next = None

# Hash Table class
class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [None] * size

    # Hash function
    def hash_function(self, key):
        return key % self.size

    # Insert key-value
    def insert(self, key, value):
        index = self.hash_function(key)
        new_node = Node(key, value)
        if self.table[index] is None:
            self.table[index] = new_node
        else:
            # Insert at beginning of linked list (chaining)
            new_node.next = self.table[index]
            self.table[index] = new_node

    # Search key
    def search(self, key):
        index = self.hash_function(key)
        current = self.table[index]
        while current:
            if current.key == key:
                return current.value
            current = current.next
        return None  # Not found

    # Delete key
    def delete(self, key):
        index = self.hash_function(key)
        current = self.table[index]
        prev = None
```

Dept of AIML

```
        while current:
            if current.key == key:
                if prev:
                    prev.next = current.next
                else:
                    self.table[index] = current.next
                return True
            prev = current
            current = current.next
        return False  # Not found

    # Display
    def display(self):
        for i, node in enumerate(self.table):
            print(f"Index {i}:", end=" ")
            current = node
            while current:
                print(f"({current.key}:{current.value})", end=" -> ")
                current = current.next
            print("None")

# Example Usage
ht = HashTable(5)
ht.insert(10, "Alice")
ht.insert(15, "Bob")
ht.insert(20, "Charlie")
ht.insert(7, "David")

print("Hash Table:")
ht.display()

print("Search key 15:", ht.search(15))
ht.delete(15)
print("After deleting key 15:")
ht.display()
```

OUTPUT:

```
Hash Table:
Index 0: (20:Charlie) -> (15:Bob) -> (10:Alice) -> None
Index 1: None
Index 2: (7:David) -> None
Index 3: None
Index 4: None
Search key 15: Bob
After deleting key 15:
```

Dept of AIML

```
Index 0: (20:Charlie) -> (10:Alice) -> None
Index 1: None
Index 2: (7:David) -> None
Index 3: None
Index 4: None
```

\

Dept of AIML